



Universidade do Minho

Algoritmos e Estruturas de Dados

Engenharia de Telecomunicações e Informática

2022/2023

Telmo Adão

Departamento de Sistemas de Informação

Universidade do Minho

Slides baseados no material didático disponibilizado pelo Professor Luís Magalhães

1



Universidade do Minho

Sumário

- Recursividade
- Tipos de estruturas (primitivas e complexas)
- Algoritmos de pesquisa
- Análise da Complexidade

2



Universidade do Minho

Funções recursivas

- Conceito de função recursiva
 - Função que se invoca a si mesma
- Exemplo da função fatorial
 - $5! = 5 * 4 * 3 * 2 * 1$
 - $5! = 5 * 4! , \quad 4! = 4 * 3! , \quad 3! = 3 * 2! \dots$
 - $n! = n * (n-1)!$
- A primeira instrução de uma função recursiva deve ser sempre a implementação da condição de paragem
- A chamada recursiva da função é sempre relativa a um subconjunto

3



Universidade do Minho

Funções recursivas

- Exercício
 - Escrever uma função recursiva para calcular o fatorial de um número n

$$\text{fat}(n) = \begin{cases} 1 & \text{se } n=0 \\ n \times \text{fat}(n-1) & \text{se } n>0 \end{cases}$$

4



Universidade do Minho

Funções recursivas

- Exercício

- Escrever uma função recursiva para calcular o fatorial de um número n

```
int fat (int n)
{
    if (n==0) /*condição de paragem */
        return 1;
    else
        return n*fat(n-1); /* invocação recursiva */
}
```

5



Universidade do Minho

Funções recursivas



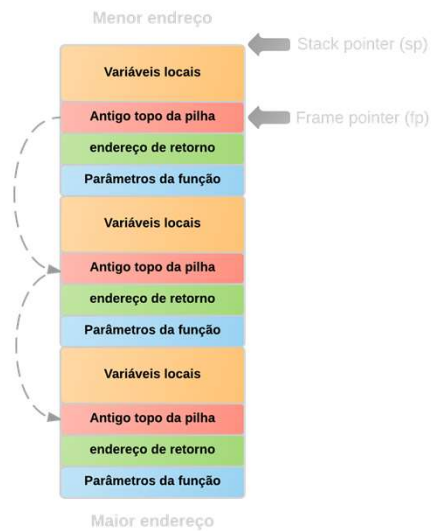
6

6



Universidade do Minho

Funções recursivas



7

7



Universidade do Minho

Recursividade vs. Iteração

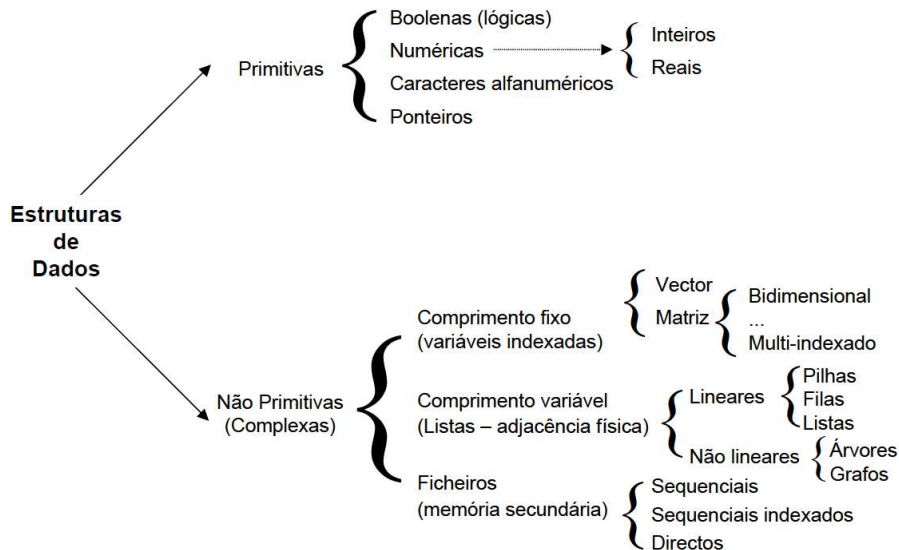
- Repetição
 - Iteration: loop explícito
 - Recursion: repete chamadas às funções, em profundidade
- Paragem
 - Iteration: condição de continuidade deixa de se verificar
 - Recursion: é reconhecido o caso base
- Ambos podem repetir-se infinitamente (cuidado, se calhar não queremos)
- A escolha deve considerar balanceamento entre performance (iteração) e boas-práticas da engenharia de SW

8



Universidade do Minho

Estruturas de Dados



9



Universidade do Minho

Estruturas de Dados - comprimento fixo

- **Vetor**
 - É um tipo estruturado que pode agrupar numa mesma variável um conjunto de valores do mesmo tipo. Um vetor é designado por um identificador, mas cada elemento tem um ou mais índices associados.
 - Unidimensional (Vetor) - 1 índice
 - Multidimensional (Matriz) - Vetor de vetores - 2+ índices
 - Cardinalidade bem definida;
 - Acesso direto ou sequencial aos elementos.

10



Estruturas de Dados - comprimento fixo

Universidade do Minho

- Vetor

- Pseudo-código

Declaração do vetor

NomeDoVetor: vetor[início..final] de tipodevariavel

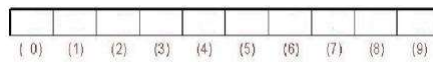
Acesso aos elementos do vetor

NomeDoVetor[índice]

Exemplo:

V: vetor[0..9] de real

V[1] <- 1



11



Universidade do Minho

Vetores (arrays) em C

Name of array (Note that all elements of this array have the same name, c

- Array
 - Group of consecutive memory locations
 - Same name and type
- To refer to an element, specify
 - Array name
 - Position number
- Format:
 - arrayname [position number]*
 - First element at position 0
 - n element array named c:
 - c[0], c[1] ... c[n-1]

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

Position number of the element within array c

12



Matrizes (vetores bidimensionais)

Universidade do Minho

– Pseudo-código

Declaração da matriz

NomeDaMatriz: vetor[linhainício..linhafinal, colunainício..colunafinal]
de tipodevariavel

Acesso aos elementos do vetor

NomeDaMatriz[linh,coluna]

Exemplo:

M: vetor[0..9,0..3] de real

M[1,2] <- 1

13



Vetores multidimensionais (arrays) em C

Universidade do Minho

- Array com várias dimensões
- Declaração

tipo **vetor**[**dim₁**] [**dim₂**] [...] [**dim_n**]

– Exemplo:

- `int a[10][2];`
- `char b[5][3][4];`

14



Universidade do Minho

Algoritmos de Pesquisa

- Algoritmos de Pesquisa em Vetores:
 - Pesquisa Sequencial
 - Pesquisa Binária

15



Universidade do Minho

Pesquisa Sequencial

- Problema (pesquisa de valor num vetor):
 - Verificar se um valor existe num vetor e, no caso de existir, indicar a sua posição.
 - Possíveis variantes para o caso de vetores com valores repetidos:
 - (a) indicar a posição da primeira ocorrência
 - (b) indicar a posição da última ocorrência
 - (c) indicar a posição de uma ocorrência qualquer

16



Universidade do Minho

Pesquisa Sequencial

- Algoritmo (pesquisa sequencial):
 - Percorrer sequencialmente todas as posições do vetor, da primeira para a última ^(a) ou da última para a primeira ^(b), até encontrar o valor pretendido ou chegar ao fim do vetor
 - ^(a) caso se pretenda saber a posição da primeira ocorrência
 - ^(b) caso se pretenda saber a posição da última ocorrência
- Adequado para vetores não ordenados ou pequenos

17



Universidade do Minho

Pesquisa Sequencial

funcao PesquisaSequencial (vet:vetor[0..n-1] de tipo, n: inteiro, alvo: tipo): inteiro

Variáveis

i: inteiro

Início

Para i <- 0 até n-1

Se (vet[i] = alvo) então

retorne i

Fim_se

Fim_para

retorne -1

Fim_funcao

18

18



Universidade do Minho

Exemplo de implementação da Pesquisa Sequencial em C

```
/* Procura um valor x num array v de n inteiros ( $n \geq 0$ ).  
Retorna o índice da primeira ocorrência de x em v, se  
encontrar; senão, retorna -1. */
```

```
int SequentialSearch(int v[], int n, int x)  
{  
    int i;  
    for (i = 0; i < n; i++)  
        if (v[i] == x) return i; /* encontrou */  
  
    return -1; /* não encontrou */  
}
```

19



Universidade do Minho

Pesquisa Sequencial

- Desafio
 - Dar como resultado as posições de todas as ocorrências do valor a procurar
 - Possíveis soluções:
 - Devolver um array de índices

20



Universidade do Minho

Pesquisa Binária

- Problema (pesquisa de valor em vetor ordenado): verificar se um valor (x) existe num vetor (v) previamente ordenado e, no caso de existir, indicar a sua posição
 - no caso de vetores com valores repetidos, consideramos a variante em que basta indicar a posição de uma ocorrência qualquer (as outras ocorrências do mesmo valor estão em posições contíguas)
- Algoritmo (pesquisa binária):
 - Comparar o valor que se encontra a meio do vetor com o valor procurado, podendo acontecer uma de três coisas:
 - é igual ao valor procurado $\Rightarrow$ está encontrado
 - é maior do que o valor procurado $\Rightarrow$ continuar a procurar (do mesmo modo) no sub-vetor à esquerda da posição inspecionada
 - é menor do que o valor procurado $\Rightarrow$ continuar a procurar (do mesmo modo) no sub-vetor à direita da posição inspecionada.
 - Se o vetor a inspecionar se reduzir a um vetor vazio, conclui-se que o valor procurado não existe no vetor inicial.

21

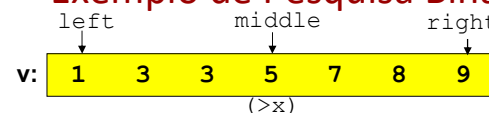


Universidade do Minho

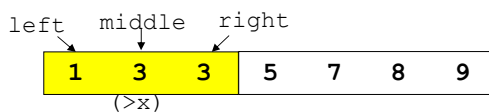
x: 2

1ª iteração

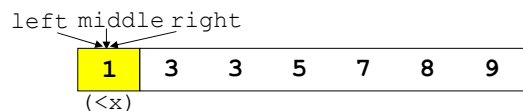
Exemplo de Pesquisa Binária



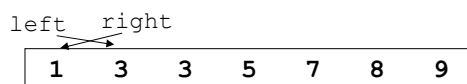
2ª iteração



3ª iteração



4ª iteração



vetor a inspecionar vazio $\Rightarrow$ o valor 2 não existe no vetor inicial!

22



Universidade do Minho

Pesquisa binária

funcao PesquisaBinaria (vet:vetor[0..n-1] de tipo, n: inteiro, alvo: tipo): inteiro

Variáveis

esq, dir, meio : inteiro

Início

esq <- 0

dir <- n-1

Enquanto (esq<=dir) fazer

meio <- (esq+dir)/2

Se (alvo = vet[meio]) então

retorne meio

Senão

Se (alvo > vet[meio]) então

esq <- meio+1

senão

dir <- meio-1

Fim_se

Fim_enquanto

retorne -1

Fim_funcao

23

23



Exemplo de implementação da Pesquisa Binária em C

Universidade do Minho

/* Procura um valor **x** num array **v** de tamanho **n** previamente ordenado. Retorna o índice de uma ocorrência de **x** em **v**, se encontrar; senão, retorna -1. Supõe que os elementos do array são comparáveis com os operadores de comparação. */

```
int BinarySearch(int v[], int n, int x)
{
    int left = 0, right = n - 1;
    int middle;
    while (left <= right)
    {
        middle = (left + right) / 2;
        if (x == v[middle]) return middle; /* encontrou */
        else if (x > v[middle]) left = middle + 1;
        else right = middle - 1;
    }
    return -1; /* não encontrou */
}
```

24



Universidade do Minho

Pesquisa binária (versão recursiva)

funcao PesquisaBinaria (vet:vetor[0..n-1] de tipo, ini: inteiro, fim: inteiro, alvo: tipo): inteiro

Variáveis

i: inteiro

Início

Se (ini>fim)

retorne -1

senão

i <- (ini+fim)/2

Se (alvo=vet[i]) então

retorne i

senão

Se (alvo > vet[i]) então

retorne PesquisaBinaria(vet,i+1,fim,alvo)

senão

retorne PesquisaBinaria(vet,ini,i-1,alvo)

Fim_se

Fim_se

Fim_se

Fim_função

25

25



Universidade do Minho

Pesquisa binária (versão recursiva)

- Exercício

- Escrever em C a versão recursiva

26

26



Universidade do Minho

Pesquisa binária (versão recursiva)

- Exercício: escrever em C a versão recursiva

```
int binarySearch(int[] A, int low, int high, int x)
{
    if (low > high) {
        return -1;
    }
    int mid = (low + high) / 2;
    if (x == A[mid]) {
        return mid;
    }
    else if (x < A[mid]) {
        return binarySearch(A, low, mid - 1, x);
    }
    else {
        return binarySearch(A, mid + 1, high, x);
    }
}
```

27

27



Universidade do Minho

Eficiência dos algoritmos de Pesquisa

- Eficiência temporal de **SequentialSearch**
 - A operação realizada mais vezes é o teste da condição de continuação do ciclo **for**, no máximo **n+1** vezes (no caso de não encontrar **x**).
 - Se **x** existir no array, o teste é realizado aproximadamente **n/2** vezes em média (1 vez no melhor caso)
 - **T(n) = O(n)** (linear) no pior caso e no caso médio
- Eficiência espacial de **SequentialSearch** e da **Pesquisa Binária**
 - Gasta o espaço das variáveis locais (incluindo argumentos)
 - Como os arrays são passados "por referência" (de facto o que é passado é o endereço do array), o espaço gasto pelas variáveis locais é constante e independente do tamanho do array
 - **S(n) = O(1)** (constante) em qualquer caso

28



Eficiência Temporal da Pesquisa Binária

Universidade do Minho

- Em cada iteração, o tamanho do sub-array a analisar é dividido por um fator de aproximadamente 2
- Ao fim de k iterações, o tamanho do sub-array a analisar é aproximadamente $n / 2^k$
- Se não existir no array o valor procurado, o ciclo só termina quando $n / 2^k \approx 1 \Leftrightarrow \log_2 n - k \approx 0 \Leftrightarrow k \approx \log_2 n$
- Assim, no pior caso, o nº de iterações é aproximadamente $\log_2 n$
 $\Rightarrow T(n) = O(\log n)$ (logarítmico)
- É muito mais eficiente que a pesquisa sequencial, mas só é aplicável a arrays ordenados!